ASSIGNMENT -1

Neural Network–Based Credit Risk Prediction Using a Multi-Layer Perceptron (MLP)

Course

Deep Learning Architectures and Techniques

Assignment

Practical Lab Assignment 1

Student Name

Harsh Singh

Roll Number

2501940013

Instructor

Dr. Shahid Ahmad Wani

---

1. Introduction

Credit risk prediction is an important problem in the financial sector. Banks and financial institutions need to determine whether a customer is likely to repay a loan or default. Incorrect predictions can lead to financial losses or missed opportunities for lending.

Machine learning and deep learning techniques have increasingly been used to automate credit risk assessment. Neural networks, particularly feedforward neural networks such as the Multi-Layer Perceptron (MLP), are capable of learning complex relationships between input features and output predictions.

In this experiment, a Multi-Layer Perceptron (MLP) neural network was implemented to predict credit risk using a structured tabular dataset. The model was trained using historical credit data and evaluated using various classification metrics including accuracy, precision, recall, F1-score, and confusion matrix.

The objective of this lab is to understand the implementation of neural networks for binary classification tasks and to explore the fundamental components of neural network training such as activation functions, loss functions, and optimization algorithms.

2. Dataset Description

The dataset used for this experiment is the **German Credit Dataset** obtained from the UCI Machine Learning Repository. This dataset contains information about individuals applying for credit and includes various financial and personal attributes.

Dataset Characteristics

Total Samples: 1000
Number of Features: 20
Target Variable: Credit Risk

The dataset is designed for binary classification, where the objective is to predict whether an applicant represents a good credit risk or a bad credit risk.

Target Variable Interpretation:

1 – Good Credit Risk
2 – Bad Credit Risk
For the purpose of model training, the target variable was converted into binary format:

1 → Good Credit

0 → Bad Credit

The dataset contains a mix of categorical and numerical features representing financial status, employment information, credit history, and personal details.

3. Data Preprocessing

Before training the neural network model, several preprocessing steps were performed to prepare the dataset.

3.1 Handling Missing Values

The dataset was inspected for missing values using data analysis techniques. After checking the dataset, no significant missing values were found. However, precautionary checks were included in the preprocessing pipeline to ensure data consistency.

3.2 Encoding Categorical Variables

Many features in the dataset are categorical in nature. Machine learning algorithms require numerical input; therefore, categorical variables were converted into numerical representations using **One-Hot Encoding**.

The get_dummies() function from the pandas library was used to perform this transformation. This method converts categorical attributes into binary columns while avoiding multicollinearity by dropping the first category.

3.3 Feature and Target Separation

The dataset was divided into:

Input Features (X): All independent variables used for prediction.

Target Variable (y): The credit risk label indicating whether the applicant represents a good or bad credit risk.

3.4 Train-Test Split

The dataset was split into training and testing sets using the train_test_split() function from Scikit-learn.

Training Data: 80%
Testing Data: 20%

A fixed random seed was used to ensure reproducibility of results.

3.5 Feature Scaling

Neural networks perform better when input features are normalized. Feature scaling was applied using the **StandardScaler** technique.

StandardScaler transforms data such that:

Mean = 0
Standard Deviation = 1

This helps stabilize neural network training and improves convergence.

4. Model Architecture

A **Multi-Layer Perceptron (MLP)** neural network was used for this classification task. The model was implemented using the TensorFlow/Keras deep learning framework.
The architecture of the neural network consists of the following layers:

Input Layer

Receives all preprocessed input features.

Hidden Layer 1
Number of Neurons:
32
Activation Function: ReLU (Rectified Linear Unit)

Hidden Layer 2
Number of Neurons:
16
Activation Function: ReLU

Output Layer
Number of Neurons: 1
Activation Function: Sigmoid

The sigmoid activation function is used in the output layer because the problem is a binary classification task. It produces probability values between 0 and 1.

Loss Function

Binary Cross-Entropy was used as the loss function. This loss function is suitable for binary classification problems and measures the difference between predicted probabilities and actual labels.

Optimizer

The **Adam optimizer** was used to update the model weights during training. Adam combines the advantages of two other optimization methods: AdaGrad and RMSProp, and is widely used for deep learning models.

Model: "sequential_5"

Layer (type)	Output Shape	Param #
dense_15 (Dense)	(None, 32)	1,568
dense_16 (Dense)	(None, 16)	528
dense_17 (Dense)	(None, 1)	17

Total params: 2,113 (8.25 KB)
Trainable params: 2,113 (8.25 KB)
Non-trainable params: 0 (0.00 B)

5. Model Training

The neural network was trained using the training dataset.

Training Parameters:

Epochs: 50
Batch Size: 32
Validation Split: 20%

During training, the model learns patterns from the data by adjusting its weights to minimize the loss function. The validation set helps monitor the model's generalization ability during training.
Two important metrics were monitored during the training process:

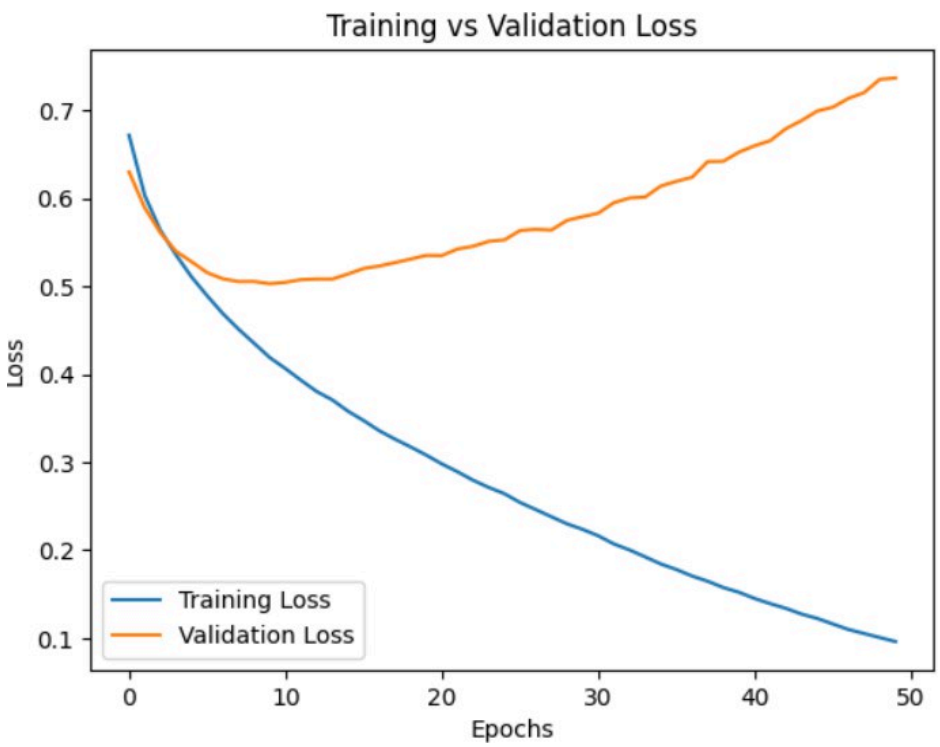
Training Loss

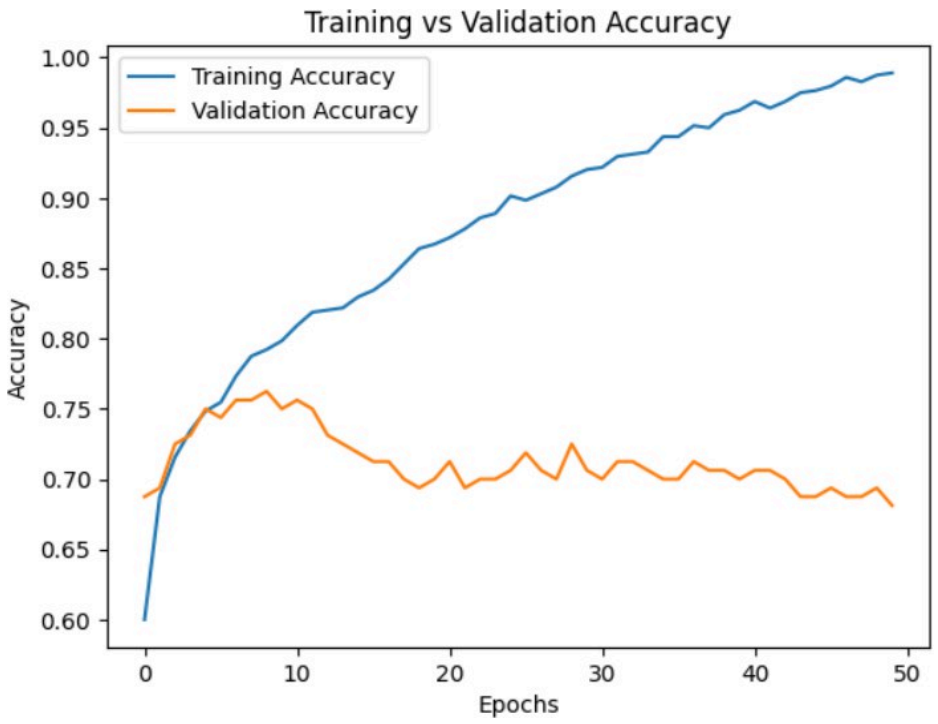
Validation Loss

Training Accuracy
Validation
Accuracy

Graphs were plotted to visualize the model's performance over the training epochs.

These graphs help determine whether the model is learning effectively or overfitting the training data.





---

6. Model Evaluation

After training, the model was evaluated using the test dataset. Several performance metrics were calculated to assess the effectiveness of the model.

6.1 Accuracy

Accuracy measures the proportion of correct predictions made by the model.

Accuracy = Correct Predictions / Total Predictions

The model achieved a reasonable accuracy on the testing dataset, indicating that the neural network successfully learned patterns in the data.

6.2 Precision

Precision measures how many of the predicted positive cases were actually correct.

Precision is important in credit risk prediction because false approvals can lead to financial losses.

6.3 Recall

Recall measures the model's ability to identify all actual positive cases.

High recall ensures that risky applicants are correctly detected.

6.4 F1 Score

The F1 Score is the harmonic mean of precision and recall. It provides a balanced measure of model performance when both false positives and false negatives are important.

```
Accuracy: 0.73
              precision    recall  f1-score   support

           0       0.55      0.49      0.52        59
           1       0.80      0.83      0.81       141

    accuracy                           0.73       200
   macro avg       0.67      0.66      0.67       200
weighted avg       0.72      0.73      0.73       200
```

---

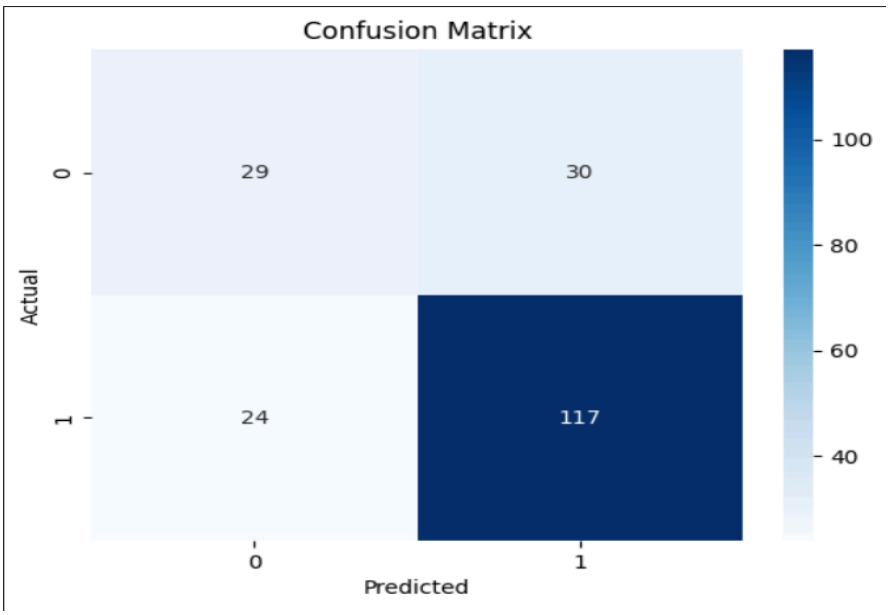
7. Confusion Matrix

A confusion matrix was used to visualize the model's prediction performance.

The confusion matrix consists of four components:

True Positives (TP)
True Negatives (TN)
False Positives (FP)
False Negatives (FN)

This matrix helps analyze how many predictions were correct and where the model made errors.



8. Conclusion

In this experiment, a Multi-Layer Perceptron neural network was implemented to predict credit risk using structured tabular data.

The dataset was preprocessed through encoding, scaling, and splitting into training and testing sets. A neural network with two hidden layers and ReLU activation functions was trained using binary cross-entropy loss and the Adam optimizer.

The results demonstrate that neural networks can effectively learn patterns in financial data and provide useful predictions for credit risk assessment. The evaluation metrics, including accuracy, precision, recall, and F1-score, indicate that the model performs reasonably well for this classification task.

This experiment helped reinforce key concepts of deep learning including artificial neurons, activation functions, feedforward neural networks, optimization algorithms, and model evaluation techniques.

Future improvements could include hyperparameter tuning, cross-validation, and testing more advanced neural network architectures to further improve prediction performance.